

```
import { Page, Locator } from
 '@playwright/test';
import { getElement } from
 '../db/elementRepo';
import { getAILocator } from
 '../ai/aiHealing';
import { saveHealingLog } from
 '../db/healingRepo';
import { getMinimalFullDOM } from
 '../../../../utils/selfHealing';
import { HealingLogger } from
 '../../../../utils/healingLogger';
import { containsDynamicId } from
 '../../../../utils/selfHealing';
import { beginFlow, logStep, endFlow } from
 '../../../../reporting/sink';

// TYPE FIRST
type FallbackResult = {
  locator: Locator;
  usedStrategy: string;
};

export async function findWithFallback(
  page: Page,
  elementName: string,
  primaryLocator: Locator,
  testName: string = 'default-test'
): Promise<FallbackResult> {
```

```

const attempted: string[] = [];

//
=====
=====
// START FLOW RECORDING (HTML report)
//
=====
=====
beginFlow(elementName, testName,
page.url());

//
=====
=====
// START LOGGING
//
=====
=====

HealingLogger.info('Test', testName);
HealingLogger.info('Element',
elementName);
HealingLogger.info('Page', page.url());

HealingLogger.info('AI SELF-HEALING
STARTED', 'Azure_OpenAI');

//
=====
=====
// STEP 1 – FETCH ELEMENT FROM DB
//
=====

```

```
=====

    HealingLogger.subSection(
        'STEP 1 – DATABASE PRIMARY LOCATOR'
    );

    const element = await
    getElement(elementName);

    if (!element) {

        HealingLogger.fail(
            `No locator data found for
            ${elementName}`
        );

        // record the dead-end before throwing
        logStep({
            tier: 'db',
            action: 'Fetch element from DB',
            status: 'failed',
            reason: `No locator data found for
            ${elementName}`,
        });
        endFlow({
            finalTier: null,
            finalLocator: null,
            healed: false,
            writtenBack: false,
        });

        throw new Error(
            `No locator data found for
            ${elementName}`
        );
    }
}
```

```
);  
}  
  
const dbPrimaryLocatorStr =  
element.primary_locator;  
  
if (dbPrimaryLocatorStr) {  
  
  HealingLogger.info(  
    'Trying',  
    dbPrimaryLocatorStr  
  );  
  
  const dbStart = Date.now();  
  
  const dbPrimary = buildLocator(  
    page,  
    dbPrimaryLocatorStr  
  );  
  
  if (  
    dbPrimary &&  
    await isValidLocator(dbPrimary)  
  ) {  
  
    HealingLogger.success(  
      `DB Primary locator worked`  
    );  
  
    logStep({  
      tier: 'db',  
      action: 'DB primary locator',  
      locator: dbPrimaryLocatorStr,  
      status: 'success',  
    });  
  }  
}
```

```
        durationMs: Date.now() - dbStart,
    });

    await saveHealingLog({
        elementName,
        testName,
        failedLocator: 'POM_PRIMARY',
        attemptedLocators: attempted,
        healedLocator: dbPrimaryLocatorStr,
        strategy: 'db-primary',
        status: 'HEALED',
        errorMessage: null,
    });

    endFlow({
        finalTier: 'db',
        finalLocator: dbPrimaryLocatorStr,
        healed: false,
        writtenBack: false,
    });

    return {
        locator: dbPrimary,
        usedStrategy: 'db-primary',
    };
}

attempted.push('db-primary');

logStep({
    tier: 'db',
    action: 'DB primary locator',
    locator: dbPrimaryLocatorStr,
    status: 'failed',
```

```

        reason: 'Locator built but matched 0
/ >1 or not visible',
        durationMs: Date.now() - dbStart,
    });

    HealingLogger.fail(
        `DB Primary locator failed`
    );
} else {

    logStep({
        tier: 'db',
        action: 'DB primary locator',
        status: 'skipped',
        reason: 'No primary_locator stored
for element',
    });
}

//
=====
// STEP 2 – SMART LOCATOR FALLBACK
//
=====

HealingLogger.subSection(
    'STEP 2 – SMART LOCATOR FALLBACK'
);

const smartLocators =
    typeof element.smart_locators ===
'string'

```

```
    ? JSON.parse(element.smart_locators)
    : element.smart_locators;

const priority = [
  'role',
  'testId',
  'label',
  'placeholder',
  'text',
  'alt',
  'title',
  'css',
  'xpath'
];

for (const key of priority) {

  const locatorStr = smartLocators?.
[key];

  if (!locatorStr) continue;

  attempted.push(key);

  HealingLogger.info(
    'Strategy',
    key
  );

  HealingLogger.info(
    'Locator',
    locatorStr
  );
}
```

```
const smartStart = Date.now();

const built = buildLocator(
  page,
  locatorStr
);

if (!built) {

  HealingLogger.fail(
    'Failed to build locator'
  );

  logStep({
    tier: 'smart',
    action: `Smart: ${key}`,
    locator: locatorStr,
    status: 'failed',
    reason: 'Failed to build locator',
    durationMs: Date.now() -
smartStart,
  });

  continue;
}

const valid = await
isValidLocator(built);

HealingLogger.info(
  'Result',
  valid ? 'SUCCESS' : 'FAILED'
);
```

```
if (valid) {

    HealingLogger.success(
        `Element healed using ${key}`
    );

    logStep({
        tier: 'smart',
        action: `Smart: ${key}`,
        locator: locatorStr,
        status: 'success',
        durationMs: Date.now() -
smartStart,
    });

    await saveHealingLog({
        elementName,
        testName,
        failedLocator: primaryLocator,
        attemptedLocators: attempted,
        healedLocator: locatorStr,
        strategy: key,
        status: 'HEALED',
        errorMessage: null,
    });

    HealingLogger.section(
        'HEALING COMPLETED'
    );

    endFlow({
        finalTier: 'smart',
        finalLocator: locatorStr,
        healed: false,
```

```

        writtenBack: false,
    });

    return {
        locator: built,
        usedStrategy: key,
    };
}

logStep({
    tier: 'smart',
    action: `Smart: ${key}`,
    locator: locatorStr,
    status: 'failed',
    reason: 'Matched 0 / >1 elements or
not visible',
    durationMs: Date.now() - smartStart,
});

//
=====
=====
// STEP 3 – AI HEALING
//
=====
=====

HealingLogger.subSection(
    'STEP 3 – AI HEALING'
);

HealingLogger.ai(
    'Collecting screenshots and DOM

```

```

context'
  );

  const aiStart = Date.now();

  try {

    //
    =====
    =====
    // CURRENT SCREENSHOT
    //
    =====
    =====

    const screenshotBuffer =
      await page.screenshot({
        fullPage: true
      });

    const currentPageScreenshot =
      screenshotBuffer.toString('base64');

    //
    =====
    =====
    // OLD ELEMENT SCREENSHOT
    //
    =====
    =====

    const oldElementScreenshot =
      element?.element_screenshot
        ? Buffer

```

```

        .from(element.element_screenshot)
        .toString('base64')
        : null;

    //
    =====
    =====
    // OLD FULL PAGE SCREENSHOT
    //
    =====
    =====

    const oldPageScreenshot =
        element?.full_page_screenshot
        ? Buffer

        .from(element.full_page_screenshot)
        .toString('base64')
        : null;

    //
    =====
    =====
    // OLD OUTER HTML
    //
    =====
    =====

    const oldOuterHTML =
        element?.outer_html || '';

    if (!oldOuterHTML) {

        HealingLogger.warn(

```

```

        'No stored outerHTML found'
    );
}

HealingLogger.ai(
    'Sending healing request to AI model'
);

//
=====
=====
// AI CALL
//
=====
=====

    const aiResponse = await getAILocator({
        elementName,
        failedLocator:
primaryLocator.toString(),
        smartLocators,
        oldOuterHTML: element.outer_html,
        oldDOM: element.full_dom,
        newDOM: await
getMinimalFullDOM(page),
        currentPageScreenshot,
        oldPageScreenshot,
        oldElementScreenshot,
        meta: element?.meta || {},
    });

//
=====
=====

```

```

// AI RESPONSE LOGGING
//
=====

HealingLogger.ai(
    `Locator      :
${aiResponse?.locator}`
);

HealingLogger.ai(
    `Confidence   :
${aiResponse?.confidence}`
);

HealingLogger.ai(
    `Reason       :
${aiResponse?.reason}`
);

//
=====

// VALIDATE AI RESPONSE
//
=====

if (
    !aiResponse ||
    !aiResponse.locator
) {

    throw new Error(

```

```

        'AI returned empty or invalid
response'
        );
    }

    //
=====
=====
    // CONFIDENCE CHECK
    //
=====
=====

    if (aiResponse.confidence < 0.7) {

        throw new Error(
            `Low confidence AI response:
${aiResponse.confidence}`
        );
    }

    //
=====
=====
    // BUILD AI LOCATOR
    //
=====
=====

    if (
        containsDynamicId(
            aiResponse.locator
        )
    ) {

```

```

        throw new Error(
            'AI returned dynamic locator'
        );
    }

    const aiLocator = buildLocator(
        page,
        aiResponse.locator
    );

    if (!aiLocator) {

        throw new Error(
            'Failed to build locator from AI
response'
        );
    }

    //
    =====
    =====
    // VALIDATE AI LOCATOR
    //
    =====
    =====

    const isValid =
        await isValidLocator(aiLocator);

    if (isValid) {

        HealingLogger.subSection(
            'HEALING SUCCESS'

```

```
);

HealingLogger.success(
    'AI locator validated successfully'
);

HealingLogger.info(
    'Final Locator',
    aiResponse.locator
);

HealingLogger.info(
    'Confidence',
    aiResponse.confidence
);

logStep({
    tier: 'ai',
    action: 'AI heal vs live DOM',
    locator: aiResponse.locator,
    status: 'success',
    reason: aiResponse.reason,
    confidence: aiResponse.confidence,
    durationMs: Date.now() - aiStart,
});

await saveHealingLog({
    elementName,
    testName,
    failedLocator:
primaryLocator.toString(),
    attemptedLocators: attempted,
    healedLocator: aiResponse.locator,
    strategy: 'AI',
```

```
        status: 'AI_HEALED',
        errorMessage: null,
    });

    HealingLogger.section(
        'HEALING COMPLETED'
    );

    endFlow({
        finalTier: 'ai',
        finalLocator: aiResponse.locator,
        healed: true,
        writtenBack: false,
    });

    return {
        locator: aiLocator,
        usedStrategy: 'AI',
    };
}

throw new Error(
    'AI locator is not valid on current
page'
);

} catch (error: any) {

    HealingLogger.subSection(
        'HEALING FAILED'
    );

    HealingLogger.fail(
        error?.message || 'Unknown healing
```

```
error'
    );

    logStep({
        tier: 'ai',
        action: 'AI heal vs live DOM',
        status: 'failed',
        reason: error?.message || 'Unknown
healing error',
        durationMs: Date.now() - aiStart,
    });

    await saveHealingLog({
        elementName,
        testName,
        failedLocator:
primaryLocator.toString(),
        attemptedLocators: attempted,
        healedLocator: null,
        strategy: 'AI',
        status: 'FAILED',
        errorMessage: error?.message ||
'Unknown error',
    });

    endFlow({
        finalTier: null,
        finalLocator: null,
        healed: false,
        writtenBack: false,
    });

    throw new Error(
        `All locator strategies including AI
```

```

failed for ${elementName}`
    );
  }
}

export function buildLocator(page: Page,
locatorStr: string): Locator | null {
  try {
    // getByRole

    if (locatorStr.startsWith('getByRole'))
    {

      const roleMatch = locatorStr.match(
        /getByRole\(\\s*['"](.*)
['"]\\s*,\\s*\\{\\s*name\\s*:\\s*['"](.*)
['"]\\s*\\}\\s*\\)/
      );

      if (roleMatch) {

        const role = roleMatch[1]?.trim();

        const name = roleMatch[2]?.trim();

        return page.getByRole(
          role as any,
          {
            name,
            exact: true,
          }
        );
      }
    }
  }
}

```

```

        console.warn(
            `Failed to parse getByRole locator:
${locatorStr}`
        );
    }

    if (
        locatorStr.startsWith(
            'getByTestId'
        )
    ) {

        const val =
            locatorStr.match(
                /getByTestId\(['"](.*)['"]\)/
            )?.[1];

        if (val) {

            return page.getByTestId(
                val
            );
        }
    }

    // getByText
    if (locatorStr.startsWith('getByText'))
    {
        const text =
            locatorStr.match(/getByText\(['"](.*)
            ['"]\)/)?.[1];
        if (text) return page.getByText(text,
            { exact: true });
    }

```

```

    }

    // getByPlaceholder
    if
    (locatorStr.startsWith('getByPlaceholder'))
    {
        const val =
        locatorStr.match(/getByPlaceholder\(['"]
        (.*)['"]\)/)?.[1];
        if (val) {
            return page.getByPlaceholder(new
            RegExp(`^${escapeRegex(val)}$`));
        }
    }

    // getByLabel
    if
    (locatorStr.startsWith('getByLabel')) {
        const val =
        locatorStr.match(/getByLabel\(['"](.*)
        ['"]\)/)?.[1];
        if (val) {
            return page.getByLabel(val, {
            exact: true });
        }
    }

    // getByAltText
    if
    (locatorStr.startsWith('getByAltText')) {
        const val =
        locatorStr.match(/getByAltText\(['"](.*)
        ['"]\)/)?.[1];
        if (val) {

```

```

        return page.getByAltText(val, {
exact: true });
    }
}

// getTitle
if
(locatorStr.startsWith('getTitle')) {
    const val =
locatorStr.match(/getTitle\(['"](.*)
['"]\)\)??.[1];
    if (val) {
        return page.getTitle(val, {
exact: true });
    }
}

// CSS / XPath fallback
return page.locator(locatorStr);

} catch {
    return null;
}
}

// Validate locator safely
export async function isValidLocator(
    locator: Locator
): Promise<boolean> {
    try {

        const count = await locator.count();

        if (count === 0) {

```

```

        HealingLogger.warn(
            'Locator matched 0 elements'
        );
        return false;
    }

    if (count > 1) {
        HealingLogger.warn(
            `Locator matched ${count} elements`
        );
        return false;
    }

    await locator.waitFor({
        state: 'visible',
        timeout: 3000,
    });

    return true;

} catch (error: any) {

    HealingLogger.warn(
        `Locator validation failed:
        ${error.message}`
    );

    return false;
}
}
function escapeRegex(text: string): string
{
    return text.replace(/[\.*+?^${}()|

```

```
[\\]\\]/g, '\\$&');  
}
```